CST8219 – C++ Programming

Lab3: Object-Oriented Student Management System

Marks: 20
Course Weight: 5%

OBJECTIVE

This lab extends Lab 2: **Advanced Student Management System** by refactoring the program into a fully object-oriented design using C++. The focus of this lab is on **class design, constructors, object lifetime, deep copy, and function behavior**.

All functionality from Lab 2 must remain operational, **except file handling**, which is **explicitly excluded** from this lab.

By completing this lab, students will demonstrate mastery of:

- Design and implement C++ classes with proper encapsulation
- Use multiple constructor types correctly
- Manage dynamic memory safely
- Implement deep and manual copying
- Apply constructor overloading and chaining
- Use destructors appropriately
- Implement virtual, overloaded, and recursive functions

Lab 3 must be implemented on top of Lab 2. Do NOT start a new project.

Pre-Requisites:

- Successful completion of **Lab 2**
- Lab 2 must be extended, not rewritten
- Menu-driven structure must be preserved
- Smart pointer usage must be retained

Major Design Change (Core Requirement):

Feature	Lab 2	Lab 3
Data storage	Memory + File	Memory only
File save/load	Required	Removed
Student representation	struct	class
Design	Procedural	Object-Oriented

Data Model (Unchanged Logic, New Design):

Each student object must contain:
- ID (int)
- Name (std::string)
- Marks in 3 subjects (dynamic array)
- Total marks (float)
- Average marks (float)
- Pass/Fail Status (enum Status)
- Letter Grade (enum class Grade)

Class Design Requirements:

Student Class (Core) – Check Appendix (student.h)

The student class must encapsulate all student-related data and behavior.

Required Attributes
- All data members must be private
- Dynamic memory must be managed using smart pointers

Required Member Functions
- Calculation of total, average, grade, and status
- Display of student information
- Safe updating of marks

CONSTRUCTOR REQUIREMENTS (Mandatory)

The Student class **must implement all the following constructor types**:

1. **Default Constructor:** Initializes safe default values
2. **Parameterized Constructor**: Accepts ID, name, and marks
3. **Copy Constructor**: Performs a **deep copy** of dynamic data
4. **Move Constructor**
   - Transfers ownership safely
   - Leaves the source object in a valid state
5. **Initializer List Constructor**: Allows creation using {} syntax for marks
6. **Delegating Constructor**: Calls another constructor to reduce duplication

DEEP & MANUAL COPY

- Dynamic memory (marks) must be copied manually
- Shallow copying is not allowed
- Copy constructor and copy assignment must:
  - Allocate new memory
  - Copy all values explicitly
- Move operations must:
  - Transfer ownership
  - Avoid memory leaks or double frees

CONSTRUCTOR OVERLOADING & CHAINING

- Multiple constructors must exist with different parameter lists
- At least one constructor **must delegate** to another
- Constructor chaining must reduce duplicate logic

DESTRUCTOR REQUIREMENTS

- Destructor must be implemented
- It must ensure safe cleanup of owned resources
- No memory leaks allowed
- Smart pointers must clean up automatically

FUNCTION BEHAVIOR REQUIREMENTS

1. Virtual Functions
   - At least one virtual function. Example: a display or summary function

```
virtual void display() const;
```

2. Function Overloading
   - At least one function must be overloaded. Same function name, different parameters:

```
void updateMarks(const float*);
void updateMarks(float, float, float);
```

3. Recursive Function
   - At least one recursive function must be implemented. Example: Summing marks, recursive validation and recursive calculation logic

```
float recursiveTotal(int index) const;
```

MENU (UNCHANGED FROM LAB 2)

Menu must remain identical, but internally call class member functions:

```
1. Add student records
2. Display all students
3. Find highest scoring student
4. Display class summary
5. Exit
```

MENU RULES

- Menu must loop until Exit is selected
- Each option must call a separate function
- Invalid input must be handled gracefully
- switch statement is required

INPUT VALIDATION (REQUIRED):

All validation rules from Lab 2 still apply:

- Student ID must be positive and unique
- Marks must be between 0 and 100
- Name must not be empty or whitespace
- Menu input must be validated
- Program must recover from invalid input safely

EDGE CASES TO HANDLE:

Scenario	Expected Behavior
No students added	Display safe message
Copying a student	Deep copy, independent memory
Moving a student	Source safely invalidated
Self-assignment	Handled safely
Multiple additions	No memory leaks
Program exit	Clean shutdown

Lab File Structure:

File	Purpose
main.cpp Menu and program flow	main.cpp Menu and program flow
Student.h Class declaration	Student.h Class declaration
Student.cpp Constructors, destructor, logic	Student.cpp Constructors, destructor, logic
CMakeLists.txt Build configuration	CMakeLists.txt Build configuration
.git Version control	.git Version control

Git Commit Message

```
git commit -m "Completed Lab 3: OOP"
```

APPENDIX: student.h (Class Design)

```
#ifndef STUDENT_H
#define STUDENT_H

#include <string>
#include <memory>
#include <initializer_list>

// Unscoped enum for pass/fail
enum Status { FAIL, PASS };

// Scoped enum for letter grade
enum class Grade { A, B, C, D, F };

class Student {
private:
    int id;
    std::string name;
    std::unique_ptr<float[]> marks;   // dynamic array of marks
    float total;
    float average;
    Status status;
    Grade grade;

    static constexpr int SUBJECT_COUNT = 3;

public:
    // ===== Constructors =====

    // Default constructor
    Student();

    // Parameterized constructor
    Student(int id, const std::string& name, const float* marks);

    // Copy constructor (deep copy required)
    Student(const Student& other);

    // Move constructor
    Student(Student&& other) noexcept;

    // Initializer-list constructor
    Student(int id, const std::string& name,
            std::initializer_list<float> marksList);

    // Delegating constructor
    Student(int id);

    // Destructor
    virtual ~Student();
```

```
    // ===== Assignment Operators =====

    // Copy assignment (deep copy)
    Student& operator=(const Student& other);

    // Move assignment
    Student& operator=(Student&& other) noexcept;

    // ===== Member Functions =====

    // Virtual function
    virtual void display() const;

    // Core logic
    void calculateResults();

    // Overloaded functions
    void updateMarks(const float* newMarks);
    void updateMarks(float m1, float m2, float m3);

    // Recursive function
    float recursiveTotal(int index) const;

    // ===== Getters =====
    int getId() const;
    std::string getName() const;
    float getAverage() const;
    Status getStatus() const;
    Grade getGrade() const;
};

#endif // STUDENT_H
```